

Docker

Deployment & DevOps Notes — Knowledge Base

1. What is Docker?

Docker is a platform for building, shipping, and running applications inside lightweight, portable containers. Containers package an application with all its dependencies, ensuring it runs the same way across different environments — solving the classic 'it works on my machine' problem.

2. Containers vs Virtual Machines

Aspect	Containers	Virtual Machines
OS	Share host OS kernel	Each VM has a full guest OS
Startup time	Seconds	Minutes
Resource overhead	Low	High
Isolation level	Process-level	Full hardware-level

3. Key Concepts

- Image — a read-only template containing application code, runtime, libraries, and dependencies.
- Container — a running instance of an image; isolated but shares the host kernel.
- Dockerfile — a script of instructions describing how to build an image.
- Registry — a storage/distribution service for images (e.g., Docker Hub, Amazon ECR).
- Volumes — persistent storage that survives beyond a container's lifecycle.

4. Example Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

5. Common Commands

```
docker build -t myapp:1.0 .
docker run -d -p 3000:3000 myapp:1.0
docker ps
docker exec -it sh
docker-compose up -d
```

6. Why It Matters

- Consistency across dev, test, and production environments.
- Efficient resource usage compared to VMs — many containers can run on one host.
- Forms the foundation for container orchestration platforms like Kubernetes.
- Enables microservices architectures by making each service easily packaged and deployed independently.